



ADEPT TECH SOLUTIONS

AI Engineer Assessment

Autonomous Data Analyst

Estimated effort: 8–10 focused hours

Candidate Assessment Brief

Prepared by: Adept Tech Solutions — Talent & Engineering

AI Engineer — Take-Home Assessment

Autonomous Data Analyst

Time expectation: ~8–10 hours of focused effort, spread across the window you're given.

Quality of thinking matters more than polish — we would rather see honest, well-reasoned work than a rushed, over-engineered solution.

1. THE SCENARIO

A dataset is attached with this brief. Your task is to build an Agent for this dataset and churn prediction model trained on it.

The end user should be able to:

- Ask exploratory questions about the dataset — feature spread, distributions, and other EDA-style queries.
- Ask the model to assess individual data points, or submit new hypothetical data points to see how they would be evaluated against the original data.
- Query churn risk for a given customer, both as it stands today and projected forward under specific feature conditions.
- Query aggregate churn risk across segments of the dataset.

2. WHAT YOU'RE BUILDING

Stage 1 — A Model as a Callable Tool

You'll be given a business dataset (a customer churn dataset). Your job:

- Clean it. We haven't told you what's wrong with it — figure that out yourself and document what you found.
- Train a model that predicts churn risk.
- Pick an evaluation metric and justify your choice in writing. There's no single "correct" metric here — we're evaluating your reasoning, not just your number.
- Expose the model as a clean, callable function — e.g. `predict_churn_risk(customer_id) -> {risk_score, top_factors}` — because in Stage 3, your agent will call this like any other tool. Don't leave it stuck in a notebook.

Stage 2 — A Streamlit Chat Interface

Build a Streamlit app where a user can type natural-language questions about the dataset, such as:

- "Which customers are most likely to churn?"
- "Does churn risk correlate with region?"
- "Show me revenue trend for high-risk customers."

Requirements:

- Real, live wiring — the app must actually call your Stage 1 model and your Stage 3 agent, not hardcoded or pre-baked answers.
- Sensible error handling — bad input, empty results, etc. should not crash the app with a raw stack trace.
- Code organized into modules (model logic, agent logic, app/UI) — not everything in one file.

Stage 2 (Optional) — Extra Marks

Build a React-based frontend with proper pages, components, and API integration. The rest of the Stage 2 requirements above still apply.

Stage 3 — The Agent

The chat interface shouldn't just call the LLM once and print whatever it says. It needs to behave like an agent.

Model behavior:

1. Plan multi-step. A question like "which customers are likely to churn, and does that correlate with region?" needs more than one step — the agent should figure out the sequence itself (call the model, then aggregate or query the actual data, then combine).
2. Use real computation as a tool, not just the model. The agent needs a way to run actual code or queries against the dataframe (e.g. a restricted execution tool) to answer questions the model alone can't — aggregations, filters, correlations, trends.
3. Self-check before answering. If a computed result errors out, is empty, or looks nonsensical, the agent should notice and retry rather than confidently reporting garbage.
4. Never invent a number. Every specific figure the agent states in its final answer must trace back to something it actually computed. This is the single most important thing we're testing — LLMs hallucinate numbers constantly, and building something that catches this is real engineering, not prompt tweaking.

Dataset behavior:

5. The user can ask questions about the dataset itself.
6. EDA functionality is surfaced to the user, letting them see what the dataset holds and understand why the model is behaving the way it is.
7. The model tool can invoke the dataset tool for better answers about features and underlying data.

How you implement the agent loop — raw function or tool-calling, a manual plan-act-check loop, or a lightweight framework — is up to you. We're not looking for a specific stack. We're looking at whether you understand why an agent needs to plan, verify, and retry, and whether you can explain the trade-offs of what you built.

3. TOOLING — EVERYTHING HERE IS FREE

You should not need to pay for anything, and we don't expect you to.

- LLM calls: Use Groq (free tier, fast inference on Llama 3.1/3.3 and Gemma models, supports native tool-calling) or OpenRouter (free-tier models available). Either is sufficient — you don't need a frontier paid model for this.
- Code-as-tool execution: Implement this yourself as a restricted execution against a controlled namespace (e.g. just the dataframe plus safe built-ins) inside your own app process. No paid sandbox service is needed or expected.
- Model training: Plain scikit-learn, run locally.

A note on rate limits

Free-tier APIs throttle requests and tokens per minute. An agent that retries or re-plans inefficiently can burn through this quickly. This is intentional — designing an efficient loop is part of the challenge, not a bug to route around with a paid tier. If you hit limits, treat that as useful information about your design, not a blocker to escalate to us.

A note on tool-calling support

Not every free or open model handles native structured tool calls cleanly. If your chosen model struggles with this, prompting for structured JSON output is a perfectly acceptable fallback — don't burn hours fighting a model limitation instead of solving the actual problem.

4. WHAT "DONE" LOOKS LIKE (MUST COMPLETE)

- ☐ A notebook with trained model and documented data-cleaning decisions and a justified metric choice
- ☐ A working Streamlit chat app, wired live to your model and agent
- ☐ An agent that plans multi-step, computes real answers (not just LLM guesses), and self-checks before responding
- ☐ Modular, organized code (not one giant file)
- ☐ A short written reflection (see Section 6)

- ☐ Git etiquette (meaningful, incremental commits) for the whole project
- ☐ Dockerization

5. STRETCH GOALS (OPTIONAL, NO CEILING)

Pick as many or as few as you like — these are genuinely open-ended:

- A second "critic" agent that checks the analyst agent's final answer against the raw data before showing it to the user, and rejects or retries if it doesn't hold up.
- Multi-turn memory — handle follow-up questions (e.g. "now break that down by product category") without the user restating context.
- Auto-generated charts as part of the answer, not just text.
- A small eval set (10–15 questions with known correct answers) with a report on accuracy and hallucination rate.
- Deploying the app somewhere reachable (Streamlit Community Cloud is free and takes ~10 minutes).
- An ngrok reverse proxy so we can reach your server directly, without running it ourselves.

There's no expectation you finish all — or any — of these. How you use this time is itself part of what we're evaluating.

6. WHAT TO SUBMIT

1. Hosted App URL. Streamlit of Ngrok.
2. GitHub repository: With all the code. Must be public so we can evaluate the work accordingly. Must have a Google Colab Notebook with EDA and Churn Model Training and Evaluation.
3. README covering: what you built, the data issues you found and how you handled them, why you chose your metric, and how your agent's planning and verification work.
4. A short reflection (half a page max): the hardest part, what you learned or had to teach yourself, and what you'd do differently with more time.
5. An honest note on your time: roughly how you spent your ~8–10 hours, and where you stopped. We would rather see this than a repo that looks suspiciously polished for the time given.

On AI tool use

You're welcome to use Claude, ChatGPT, Copilot, or similar tools while building this — it's an AI engineering role, and pretending otherwise would test the wrong thing. Just disclose what you used it for in your README, and be ready to explain any part of your own submission in a follow-up conversation.

Good luck — we're looking forward to seeing how you think, not just what you ship.